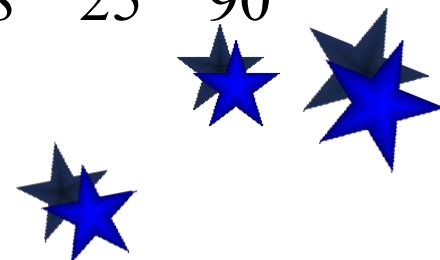


2.2 分治算法的分析技术

■ 举例2：快速排序

- 基本思想：用首元素 x 作划分标准，将输入数组 A 划分成不超过 x 的元素构成的数组 A_L ，大于 x 的元素构成的数组 A_R 。
- 递归地对子问题 A_L 和 A_R ，进行排序，直到子问题规模为1时停止。
- 划分实例：

27 99 0 8 13 64 86 16 7 10 88 25 90



2.2 分治算法的分析技术

- 一次划分的过程:

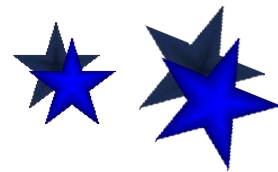
27 **99** 0 8 13 64 86 16 7 10 88 **25** 90
i *j*

27 25 0 8 13 **64** 86 16 7 **10** 88 99 90
i *j*

27 25 0 8 13 10 **86** 16 **7** 64 88 99 90
i *j*

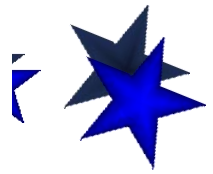
27 25 0 8 13 10 7 **16** **86** 64 88 99 90
j *i*

16 25 0 8 13 10 7 **27** 86 64 88 99 90



2.2 分治算法的分析技术

- 划分算法: **Partition (A, p, r)**
 1. $x \leftarrow A[p]$
 2. $i \leftarrow p$
 3. $j \leftarrow r + 1$
 4. while true do
 5. repeat $j \leftarrow j - 1$
 6. until $A[j] \leq x$ // 不超过首元素的
 7. repeat $i \leftarrow i + 1$
 8. until $A[i] > x$ // 比首元素大的
 9. if $i < j$
 10. then $A[i] \leftrightarrow A[j]$
 11. else return j



2.2 分治算法的分析技术

- 快速排序算法：

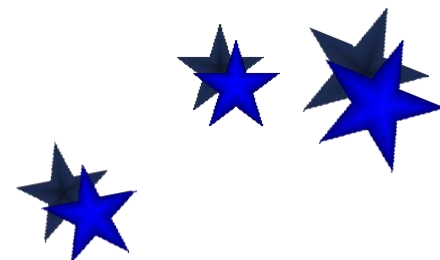
算法 Quicksort (A, p, r)

输入：数组 $A[p..r]$

输出：排好序的数组 A

1. if $p < r$
2. then $q \leftarrow \text{Partition}(A, p, r)$
3. $A[p] \leftrightarrow A[q]$
4. Quicksort ($A, p, q-1$)
5. Quicksort ($A, q+1, r$)

初始置 $p=1, r=n$ ，然后调用上述算法

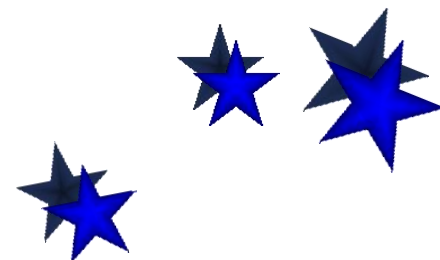


2.2 分治算法的分析技术

- 时间复杂度分析:

最坏情况: $W(n) = W(n-1) + n - 1$
 $W(1) = 0$
 $W(n) = n(n-1)/2$

最好划分: $T(n) = 2 T(n/2) + n - 1$
 $T(1) = 0$
 $T(n) = \Theta(n \log n)$



2.2 分治算法的分析技术

平均时间复杂度分析:

首元素排好序后可能放在 $1, 2, \dots, n$
各种情况概率都为 $1/n$

首元素在位置 1 : $T(0), T(n-1)$

首元素在位置 2 : $T(1), T(n-2)$

....

首元素在位置 $n-1$: $T(n-2), T(1)$

首元素在位置 n : $T(n-1), T(0)$

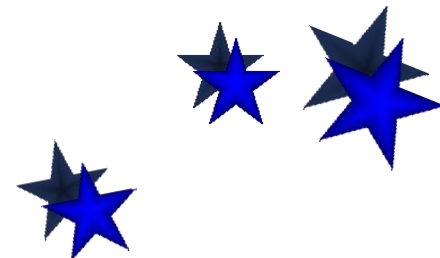
$$T(n) = \frac{2}{n} \sum_{k=1}^{n-1} T(k) + n - 1$$

$$T(1) = 0$$

$$T(n) = \Theta(n \log n)$$

子问题工作量 $2[T(1)+T(2)+\dots+T(n-1)]$

划分工作量 $n-1$



2.2 分治算法的分析技术

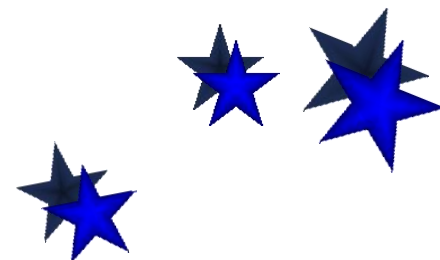
均衡划分的时间复杂度

(1) 均衡划分：每次划分后，子问题1和子问题2的规模比不变。

例如为 1:9

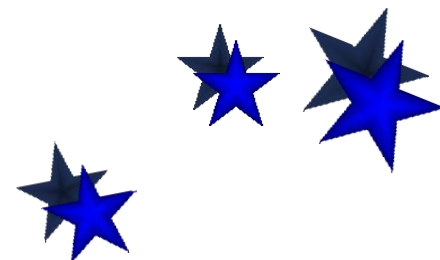
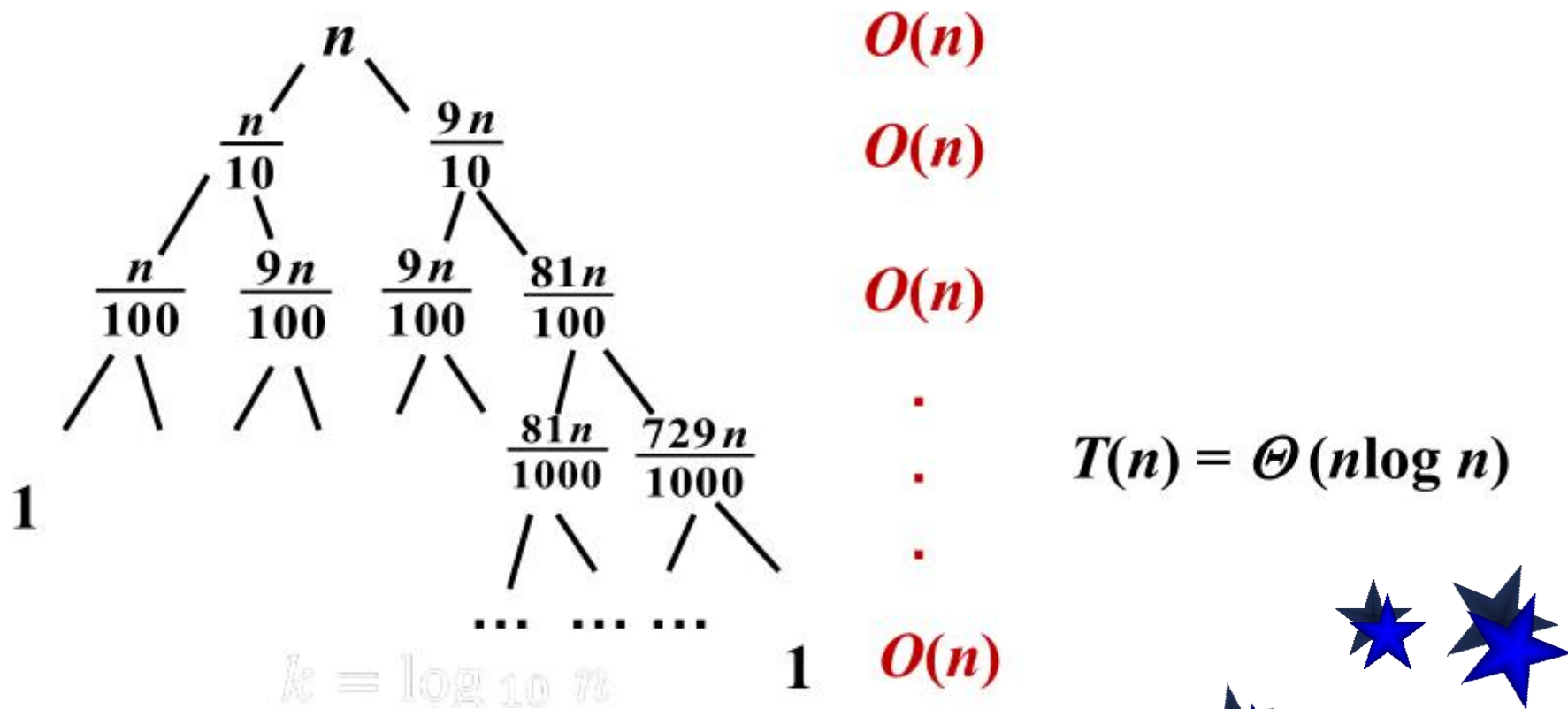
$$T(n) = T(n/10) + T(9n/10) + n$$

$$T(1) = 0$$



2.2 分治算法的分析技术

使用递归树法求解



2.2 分治算法的分析技术

■ 举例3：幂乘算法

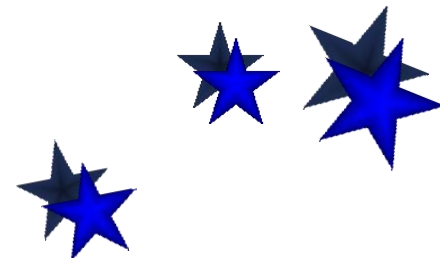
□ 输入： a 为给定实数， n 为自然数

□ 输出： a^n

□ 传统算法：顺序相乘

• $a^n = (...(((a*a)a)a)...)a$

• 基本运算：乘法次数 $\Theta(n)$



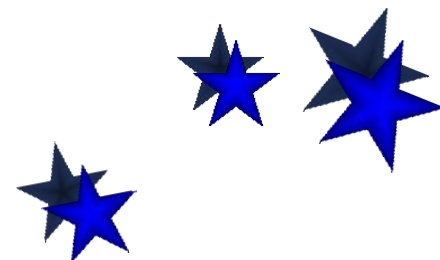
2.2 分治算法的分析技术

□ 分治算法——划分

n 为偶数 $\underbrace{a \dots a}_{n/2 \text{个}} \mid \underbrace{a \dots a}_{n/2 \text{个}}$

n 为奇数 $\underbrace{a \dots a}_{(n-1)/2 \text{个}} \mid \underbrace{a \dots a}_{(n-1)/2 \text{个}} \mid a$

$$a^n = \begin{cases} a^{n/2} \times a^{n/2} & n \text{ 为偶数} \\ a^{(n-1)/2} \times a^{(n-1)/2} \times a & n \text{ 为奇数} \end{cases}$$



2.2 分治算法的分析技术

- 分治策略算法描述

算法: **MiCheng**(a, n)

输入: a, n

输出: a^n

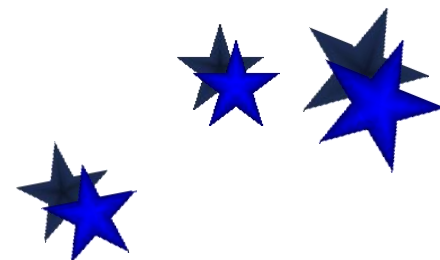
1. if $n=1$ then $y \leftarrow a$
2. else
3. $y \leftarrow$ **MiCheng**($a, n/2$)
4. $y \leftarrow y * y$
5. if(n 是奇数) then $y = y * a$
6. return(y)

以乘法作为基本运算

- 子问题规模: 不超过 $n/2$
- 两个规模近似 $n/2$ 的子问题完全一样, 只要计算1次

$$W(n) = W(n/2) + \Theta(1)$$

$$W(n) = \Theta(\log n)$$



2.2 分治算法的分析技术

□ 幂乘算法的应用——求Fibonacci 数列的第 n 项

- Fibonacci 数列: 1, 1, 2, 3, 5, 8, 13, 21, ...

满足 $F_n = F_{n-1} + F_{n-2}$

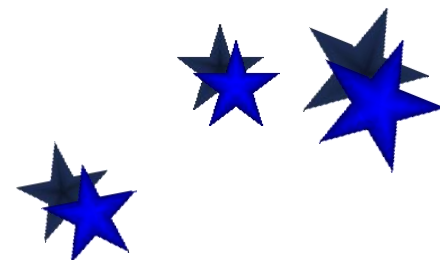
增加 $F_0=0$, 得到数列: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

问题: 已知 $F_0=0$, $F_1=1$, 给定 n , 计算 F_n

通常算法: 从 $F_0, F_1, \dots$ 开始, 根据递推公式

$F_n = F_{n-1} + F_{n-2}$, 陆续相加可得 F_n

时间为: $\Theta(n)$



2.2 分治算法的分析技术

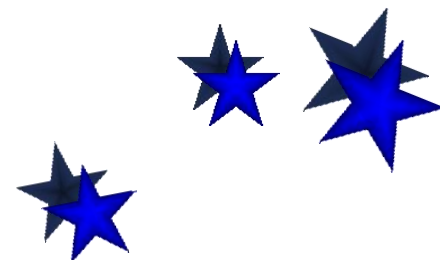
□ Fibonacci 数列的性质

- 定理1: 设 $\{F_n\}$ 为 Fibonacci 数构成的数列, 那么

$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

归纳法证明

$$n=1, \text{ 左边} = \begin{bmatrix} F_2 & F_1 \\ F_1 & F_0 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \text{右边}$$



2.2 分治算法的分析技术

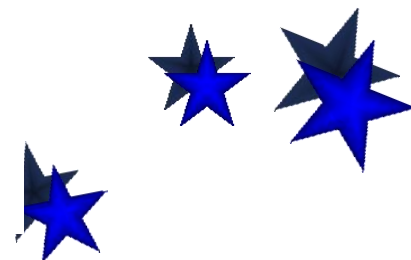
假设对任意正整数 n ，命题成立，即

$$\begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n$$

那么 $\begin{bmatrix} F_{n+2} & F_{n+1} \\ F_{n+1} & F_n \end{bmatrix} = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$

$$= \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n+1}$$

归纳假设代入



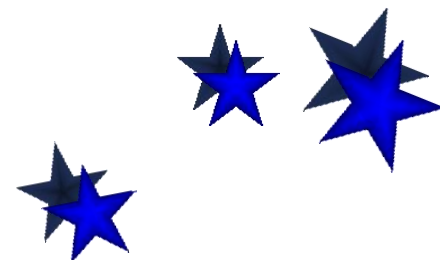
2.2 分治算法的分析技术

算法实现

令矩阵 $M = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$, 用乘幂算法计算 M^n

时间复杂度:

- 矩阵乘法次数 $T(n) = \Theta(\log n)$
- 每次矩阵乘法需要做 8 次元素相乘
- 总计元素相乘次数为 $\Theta(\log n)$



2.3 改进分治算法的途径

- 途径1：通过代数变换减少子问题的个数

- 减少子问题个数的依据

分治算法的时间复杂度方程

$$W(n) = aW(n/b) + d(n)$$

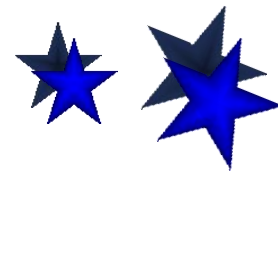
a : 子问题数, n/b : 子问题规模,

$d(n)$: 划分与综合工作量.

当 a 较大, b 较小, $d(n)$ 不大时, 方程的解:

$$\underline{W(n) = \Theta(n^{\log_b a})}$$

减少 a 是降低函数 $W(n)$ 的阶的途径。



2.3 改进分治算法的途径

- 具体方法：利用子问题的依赖关系，使某些子问题的解通过组合其他子问题的解而得到。

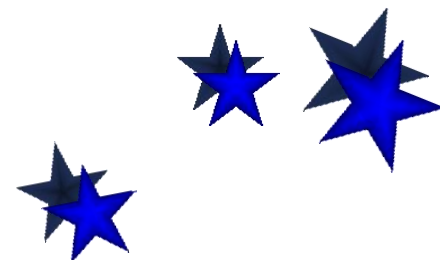
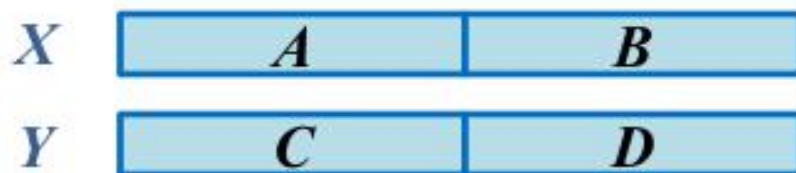
例1：整数位乘问题

输入： X, Y 是 n 位二进制数， $n = 2^k$

输出： XY

普通乘法：需要 $O(n^2)$ 次位乘运算

简单划分：令



2.3 改进分治算法的途径

$$X = A2^{n/2} + B, \quad Y = C2^{n/2} + D.$$

$$XY = \underline{AC} 2^n + (\underline{AD} + \underline{BC}) 2^{n/2} + \underline{BD}$$

说明：原 $n*n$ 的问题被划分为

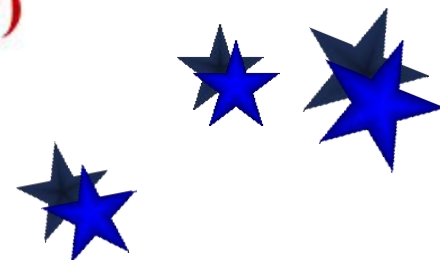
(1) 4个 $n/2$ 规模的子问题： $4W(n/2)$

(3) 3个加法运算，每个加法的时间效率为 $O(n)$

总时间效率公式为：

$$W(n) = 4W(n/2) + O(n) \Rightarrow W(n) = O(n^2)$$

分治后的策略并没有提高算法的效率。



2.3 改进分治算法的途径

- 通过代数变换，减少子问题个数

$$AD+BC = (\underline{A-B})(\underline{D-C}) + \underline{AC} + \underline{BD}$$

说明：原 $n*n$ 的问题被划分为

(1) 3个 $n/2$ 规模的子问题： $3W(n/2)$

(3) 6个加法/减运算，每个加/减法的时间效率为 $O(n)$

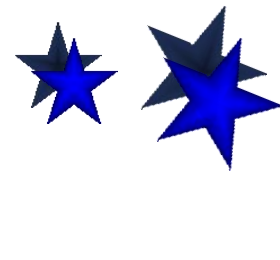
总时间效率公式为：

$$W(n) = 3 W(n/2) + cn$$

$$W(1) = 1$$

方程的解

$$W(n) = O(n^{\log 3}) = O(n^{1.59})$$



2.3 改进分治算法的途径

例2：矩阵相乘问题

输入： A, B 为 n 阶矩阵， $n = 2^k$

输出： $C = AB$

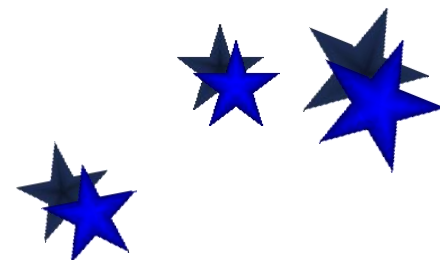
通常矩阵乘法：

C 中有 n^2 个元素

每个元素需要做 n 次乘法

以元素相乘为基本运算

$$W(n) = O(n^3)$$



2.3 改进分治算法的途径

- 矩阵相乘问题的简单分治策略

分治法 将矩阵分块，得

$$\begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

其中

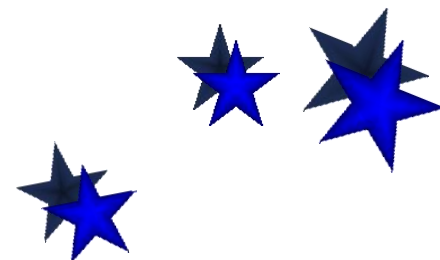
$$\begin{aligned} C_{11} &= \underline{A_{11}B_{11}} + \underline{A_{12}B_{21}} & C_{12} &= \underline{A_{11}B_{12}} + \underline{A_{12}B_{22}} \\ C_{21} &= \underline{A_{21}B_{11}} + \underline{A_{22}B_{21}} & C_{22} &= \underline{A_{21}B_{12}} + \underline{A_{22}B_{22}} \end{aligned}$$

递推方程 $W(n) = 8 W(n/2) + cn^2$

$$W(1) = 1$$

解

$$W(n) = O(n^3).$$



2.3 改进分治算法的途径

- Strassen 矩阵乘法

变换方法:

设计 $M_1, M_2, \dots, M_7$, 对应7个子问题

$$M_1 = A_{11} (B_{12} - B_{22})$$

$$M_2 = (A_{11} + A_{12}) B_{22}$$

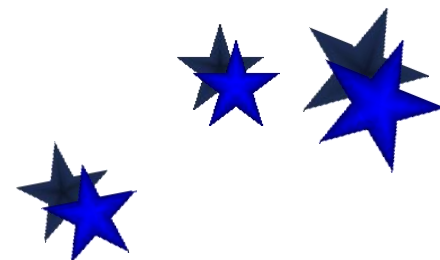
$$M_3 = (A_{21} + A_{22}) B_{11}$$

$$M_4 = A_{22} (B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{22}) (B_{11} + B_{22})$$

$$M_6 = (A_{12} - A_{22}) (B_{21} + B_{22})$$

$$M_7 = (A_{11} - A_{21}) (B_{11} + B_{12})$$



2.3 改进分治算法的途径

利用中间矩阵，得到结果矩阵

$$C_{11} = M_5 + M_4 - M_2 + M_6$$

$$C_{12} = M_1 + M_2$$

$$C_{21} = M_3 + M_4$$

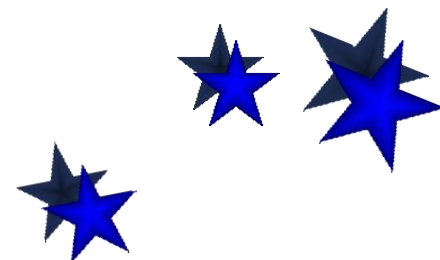
$$C_{22} = M_5 + M_1 - M_3 - M_7$$

时间复杂度函数：

$$W(n) = 7 W(n/2) + 18(n/2)^2$$

$$W(1) = 1$$

解 $W(n) = O(n^{\log 7}) = O(n^{2.8075})$



2.3 改进分治算法的途径

- 矩阵乘法的研究及应用

矩阵乘法问题的难度：

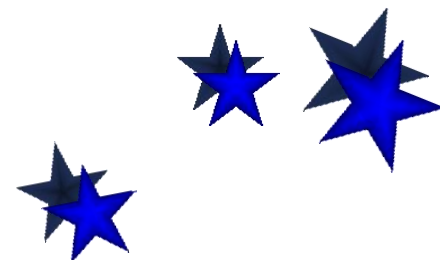
- **Coppersmith–Winograd算法：** $O(n^{2.376})$

目前为止最好的上界

- 目前为止最好的下界是： $\Omega(n^2)$

应用：

- 科学计算、图像处理、数据挖掘等
- 回归、聚类、主成分分析、决策树等挖掘算法常涉及大规模矩阵运算



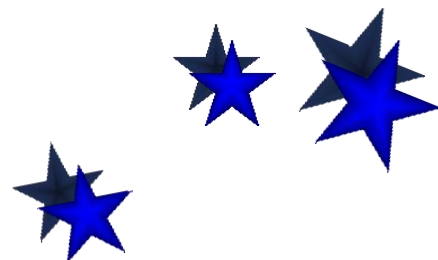
2.3 改进分治算法的途径

➤ 改进途径1小结

- 适用于：子问题个数多，划分和综合工作量不太大，时间复杂度函数

$$W(n) = \Theta(n^{\log_b a})$$

- 利用子问题依赖关系，用某些子问题解的代数表达式表示另一些子问题的解，减少独立计算子问题个数.
- 综合解的工作量可能会增加，但增加的工作量不影响 $W(n)$ 的阶.



2.3 改进分治算法的途径

■ 途径2：利用预处理减少递归内部操作

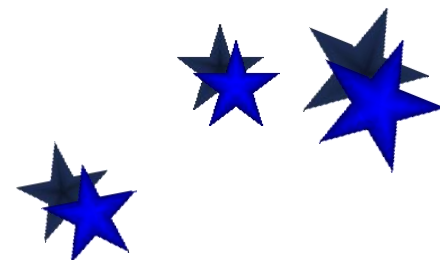
- 算法中的处理尽可能提到递归外面作为预处理
- 举例：平面点对问题

输入：平面点集 P 中有 n 个点, $n > 1$

输出： P 中的两个点，其距离最小

蛮力算法：

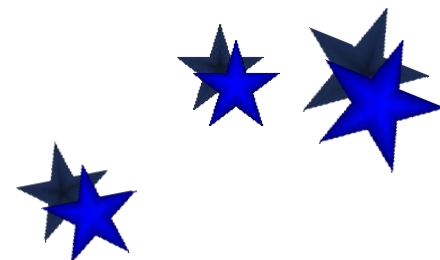
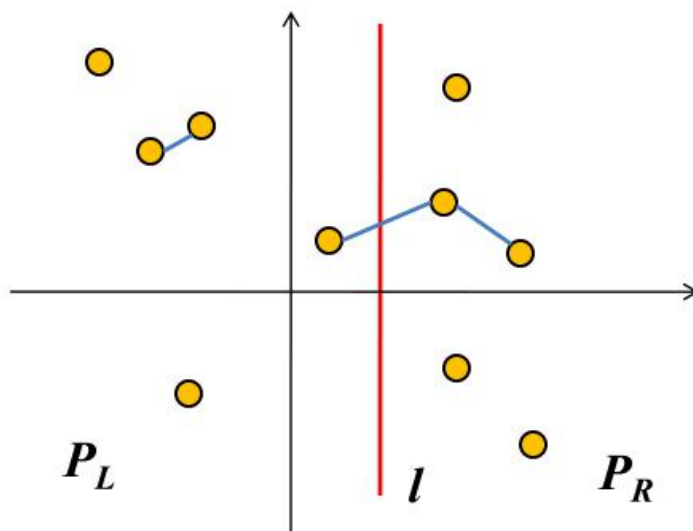
$C(n, 2)$ 个点对, 计算最小距离, $O(n^2)$



2.3 改进分治算法的途径

- 分治策略: P 划为大小相等的 P_L 和 P_R (利用横坐标的中位数)
 1. 分别计算 P_L 、 P_R 中最近点对
 2. 计算 P_L 与 P_R 中各一个点的最近点对
 3. 上述情况下的最近点对就是整个问题的解

划分实例:
 $n=10$



2.3 改进分治算法的途径

- 平面点对问题算法描述: $\text{MinDistance}(P, X, Y)$

输入: n 个点的集合 P , X 和 Y 分别为横、纵坐标数组

输出: 最近的两个点及距离

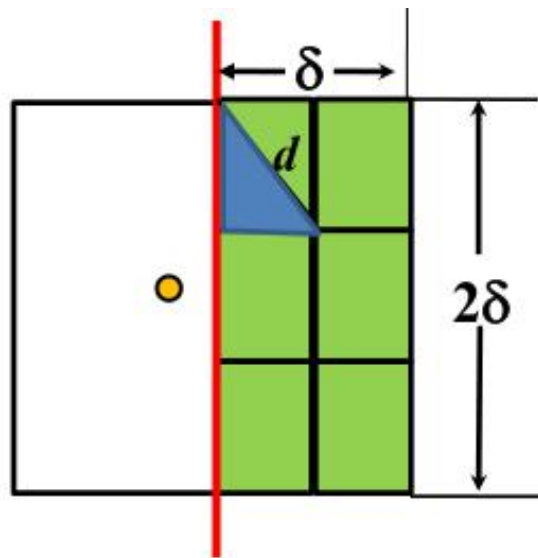
1. 如果 P 中点数小于等于3, 则直接计算其中的最小距离
2. 排序 X, Y
3. 做垂直线 l 将 P 划分为 P_L 和 P_R , P_L 的点在 l 左边, P_R 的点在 l 右边
4. $\delta_L = \text{MinDistance}(P_L, X_L, Y_L)$; P_L 中的最小距离
5. $\delta_R = \text{MinDistance}(P_R, X_R, Y_R)$; P_R 中的最小距离
6. $\delta = \min(\delta_L, \delta_R)$
7. 对于在 l 线左边距离 δ 内每个点, 检查右边是否有与之距离小于 δ 的点, 如果存在则将 δ 修改为新值



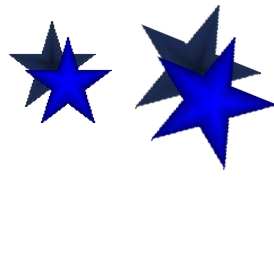
2.3 改进分治算法的途径

- 跨边界处理

$$\begin{aligned}d &= \sqrt{(\delta/2)^2 + (2\delta/3)^2} \\&= \sqrt{\delta^2/4 + 4\delta^2/9} \\&= \sqrt{25\delta^2/36} = 5\delta/6\end{aligned}$$



右边每个小方格至多1个点，每个点至多比较对面的6个点，检查1个点是常数时间， $O(n)$ 个点需要 $O(n)$ 。



2.3 改进分治算法的途径

- 算法分析

步1 递归边界处理: $O(1)$

步2 排序: $O(n\log n)$

步3 划分: $O(1)$

步4-5子问题: $2T(n/2)$

步6确定 δ : $O(1)$

步7检查跨边界点对: $O(n)$

$$T(n) = 2T(n/2) + O(n\log n)$$

$$T(n) = O(1), n \leq 3$$

不能使用主方法求解，不符合情形3

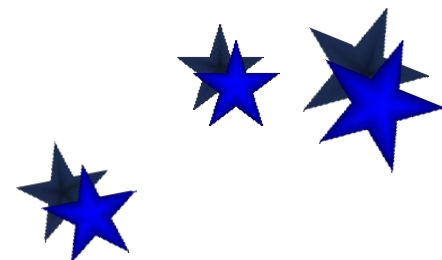
递归树求解 $T(n) = O(n\log^2 n)$



2.3 改进分治算法的途径

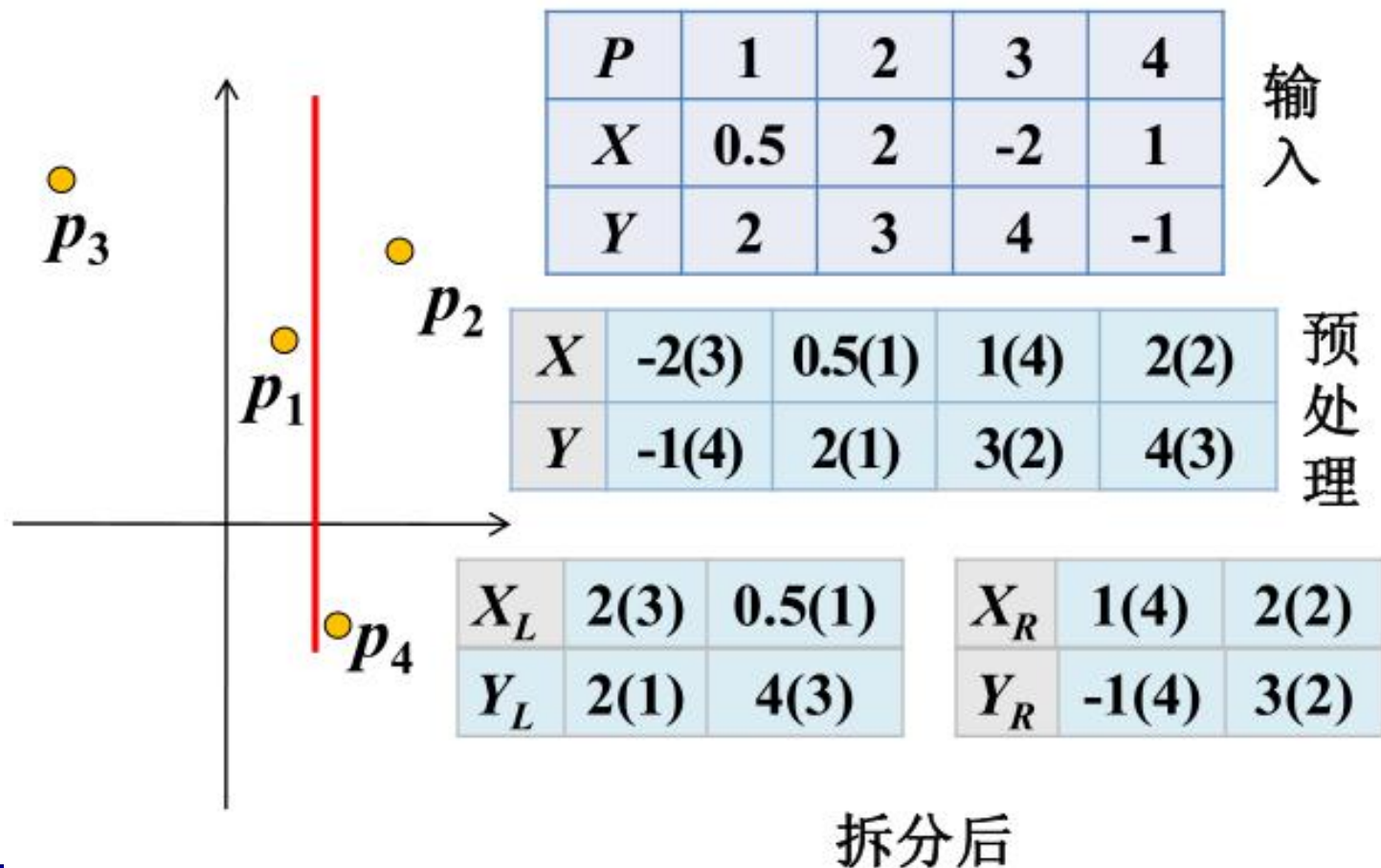
- 改进方法：增加预处理
 - (1) 原算法效率低的原因：在每次划分时对子问题数组重新排序。
 - (2) 改进策略
 - * 在递归前对 X, Y 排序，作为预处理
 - * 划分时对排序的数组 X, Y 进行拆分，得到针对子问题 P_L 的数组 X_L, Y_L 针对子问题 P_R 的数组 X_R, Y_R

原问题规模为 n ，拆分的时间为 $O(n)$



2.3 改进分治算法的途径

(3) 实例：递归中的拆分



2.3 改进分治算法的途径

(4) 改进算法时间复杂度

$W(n)$ 为算法时间复杂度

递归过程: $T(n)$, 预处理: $O(n\log n)$

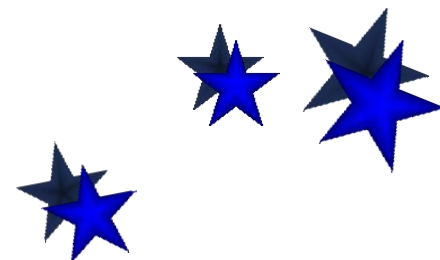
$$W(n) = T(n) + O(n\log n)$$

$$T(n) = 2T(n/2) + O(n)$$

$$T(n) = O(1) \quad n \leq 3$$

解得 $T(n) = O(n\log n)$

于是 $W(n) = O(n\log n)$



2.3 改进分治算法的途径

■ 小结

- 依据

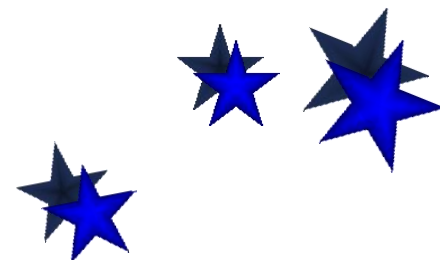
$$W(n) = aW(n/b) + f(n)$$

- 提高算法效率的方法:

- 减少子问题个数 a :

$$W(n) = O(n^{\log_b a})$$

- 增加预处理, 减少 $f(n)$



2.4 典型实例

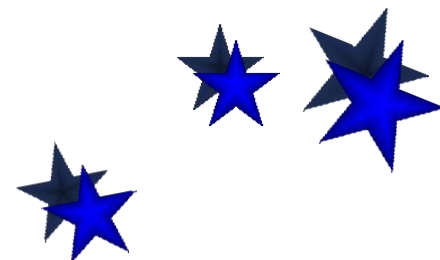
■ 选择问题

- 选最大
- 选最大最小
- 选第二大
- 选第 k 小

■ 信号平滑处理

- 卷积及应用
- 卷积计算
- 快速傅立叶变换FFT算法

■ 计算几何：计算平面点集的凸包



2.4 典型实例

■ 选择问题

输入：集合 L (含 n 个不等的实数)

输出： L 中第 i 小元素

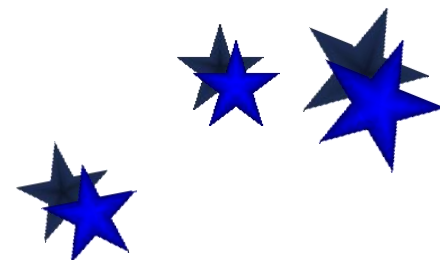
$i=1$, 称为最小元素

$i=n$, 称为最大元素

位置处在中间的元素, 称为中位元素

n 为奇数, 中位数唯一, $i = (n+1)/2$

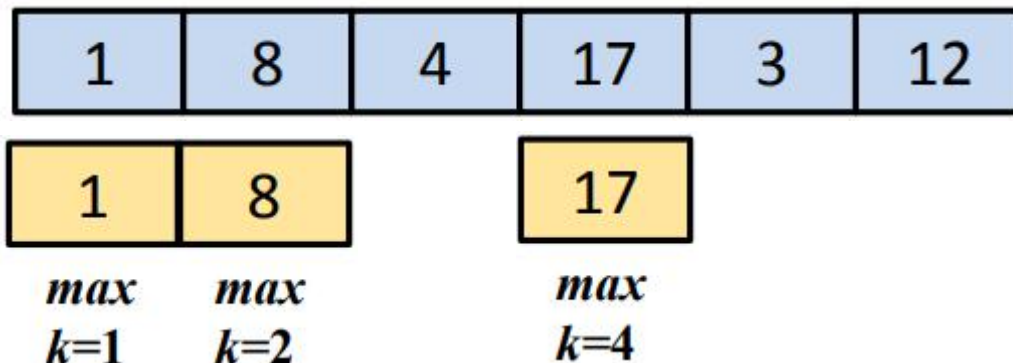
n 为偶数, 可指定 $i = n/2+1$



2.4 典型实例

□ 选最大

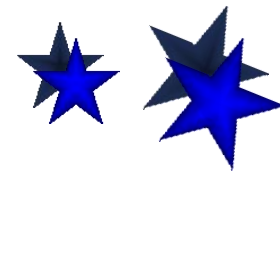
- 实现：顺序比较



输出： $\text{max} = 17$, $k=4$

算法最坏情况下的时间 $W(n)=n-1$

最好效率



2.4 典型实例

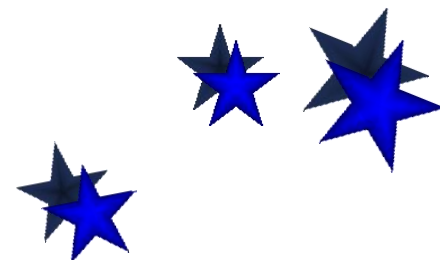
- 算法实现

算法 Findmax

输入: n 个数的数组 L

输出: max, k

1. $max \leftarrow L[1]$
2. for $i \leftarrow 2$ to n do
3. if $max < L[i]$
4. then $max \leftarrow L[i]$
5. $k \leftarrow i$
6. return max, k



2.4 典型实例

□ 选最大最小

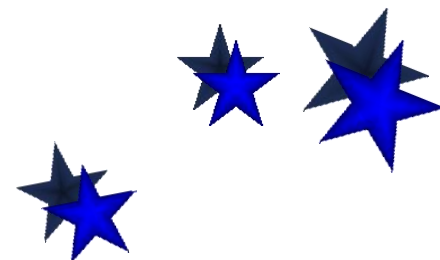
- 实现算法1：顺序比较

1. 顺序比较，先选最大 *max*

2. 顺序比较，在剩余数组中选最小 *min*，类似于选最大算法，但比较时保留较小的数

时间复杂性：

$$W(n) = n-1 + n-2 = 2n-3$$



2.4 典型实例

- 实现算法2：分组算法

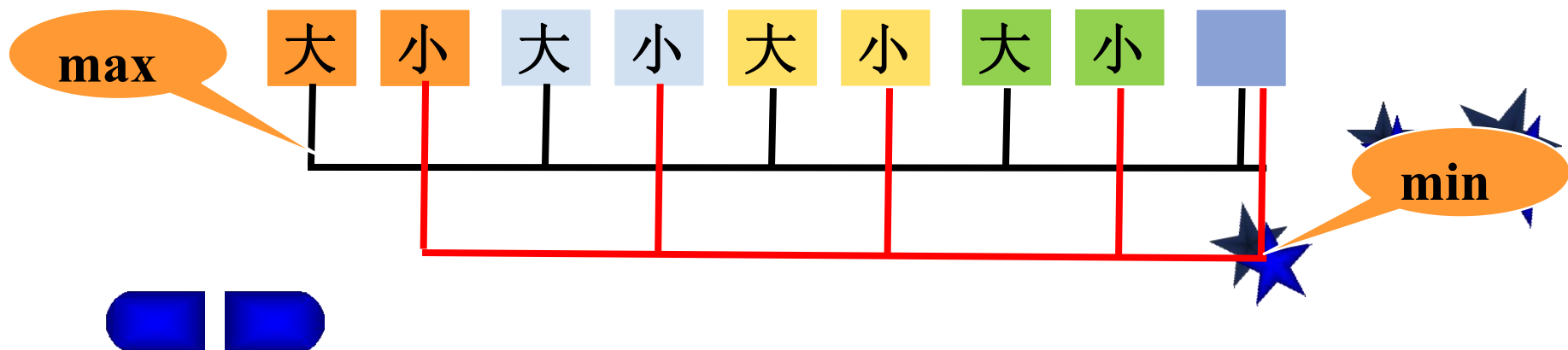
(1) 将所有的数两两分到一组，剩余的一个元素轮空



(2) 比较每组的两个数，大数放在前，小数放在后



(3) 在所有大及轮空中选最大，在所有小及轮空中选小



2.4 典型实例

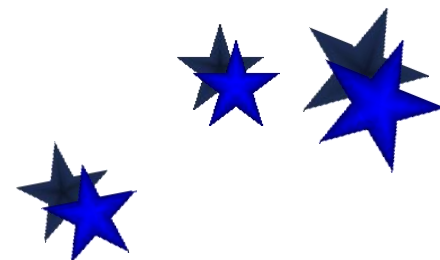
- 算法实现

算法 FindMaxMin

输入： n 个数的数组 L

输出： max , min

1. 将 n 个元素两两一组分成 $\lfloor n/2 \rfloor$ 组
2. 每组比较，得到 $\lfloor n/2 \rfloor$ 个较小和 $\lfloor n/2 \rfloor$ 个较大
3. 在 $\lceil n/2 \rceil$ 个较大（含轮空元素）中找最大 max
4. 在 $\lceil n/2 \rceil$ 个较小（含轮空元素）中找最小 min



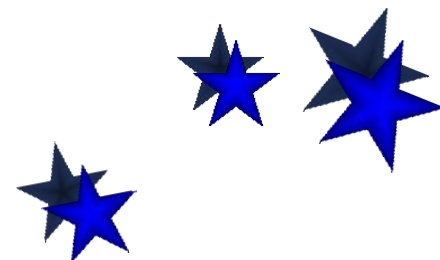
2.4 典型实例

- 最坏情况时间复杂度

行2 的组内比较: $\lfloor n/2 \rfloor$ 次

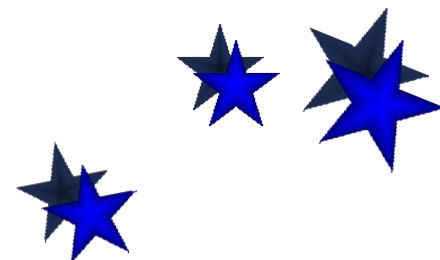
行3--4 求 *max* 和 *min* 比较:
至多 $2\lceil n/2 \rceil - 2$ 次

$$\begin{aligned} W(n) &= \lfloor n/2 \rfloor + 2\lceil n/2 \rceil - 2 \\ &= n + \lceil n/2 \rceil - 2 \\ &= \lceil 3n/2 \rceil - 2 \end{aligned}$$



2.4 典型实例

- 实现算法3：分治算法
 1. 将数组 L 从中间划分为两个子数组 L_1 和 L_2
 2. 递归地在 L_1 中求最大 max_1 和 min_1
 3. 递归地在 L_2 中求最大 max_2 和 min_2
 4. $max \leftarrow \max\{max_1, max_2\}$
 5. $min \leftarrow \min\{min_1, min_2\}$



2.4 典型实例

- 最坏情况时间复杂度

假设 $n = 2^k$,

$$W(n) = 2W(n/2) + 2$$

$$W(2) = 1$$

解 $W(2^k) = 2W(2^{k-1}) + 2$

$$= 2[2W(2^{k-2}) + 2] + 2$$

$$= 2^2 W(2^{k-2}) + 2^2 + 2 = \dots$$

$$= 2^{k-1} + 2^{k-1} + \dots + 2^2 + 2$$

$$= 3 \cdot 2^{k-1} - 2 = 3n/2 - 2$$

最好效率



2.4 典型实例

□ 选第二大

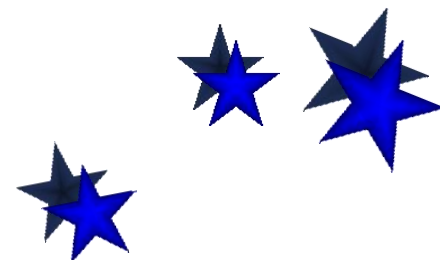
输入： n 个数的数组 L

输出： 第二大的数 $second$

- 实现算法1： 顺序比较
 1. 顺序比较找到最大 max
 2. 从剩下 $n-1$ 个数中找最大，就是第二大 $second$

时间复杂度：

$$W(n) = n - 1 + n - 2 = 2n - 3$$



2.4 典型实例

- 提高效率的途径

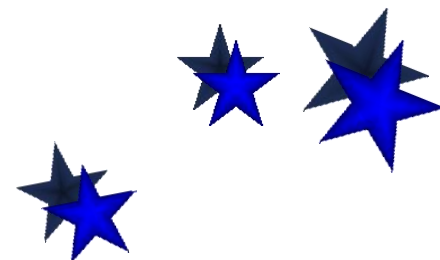
(1) 成为第二大数的条件：仅在与最大数的比较中被淘汰。

(2) 要确定第二大数，必须知道最大数。

(3) 在确定最大数的过程中记录下被最大数直接淘汰的元素。

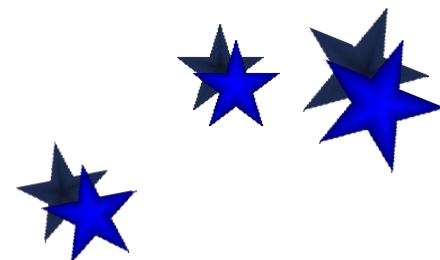
(4) 在被最大数直接淘汰的数中找到的最大数就是第二大数。

设计思想：用空间换时间



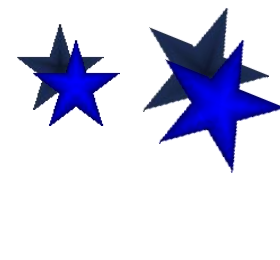
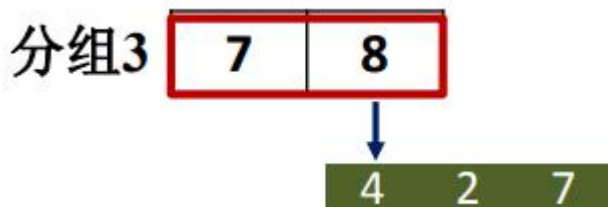
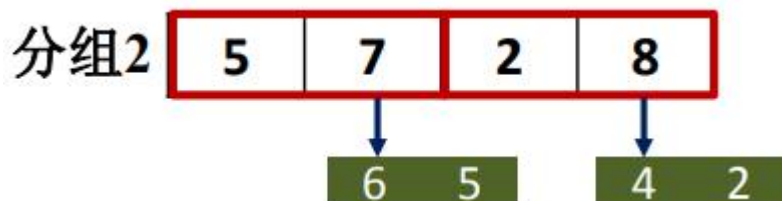
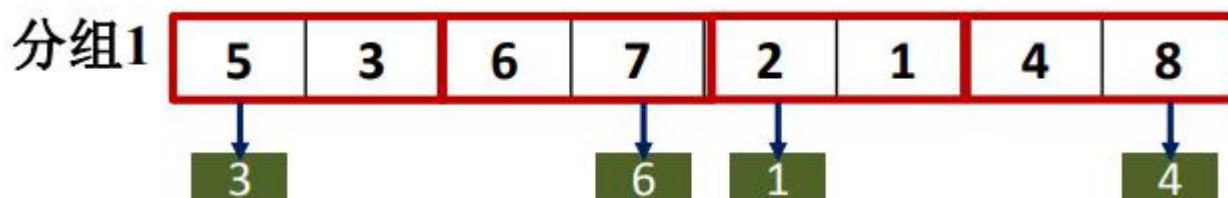
2.4 典型实例

- 实现算法2：锦标赛算法
 1. 两两分组比较，大者进入下一轮，直到剩下 1 个元素 *max* 为止
 2. 在每次比较中淘汰较小元素，将被淘汰元素记录在淘汰它的元素的链表上
 3. 检查 *max* 的链表，从中找到最大元，即 *second*



2.4 典型实例

实例演示



2.4 典型实例

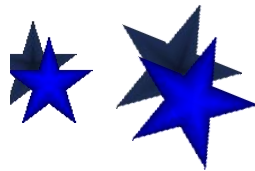
算法 FindSecond

输入: n 个数的数组 L , 输出: $second$

1. $k \leftarrow n$ // 参与淘汰的元素数
2. 将 k 个元素两两1组, 分成 $\lfloor k/2 \rfloor$ 组
3. 每组的2个数比较, 找到较大数
4. 将被淘汰数记入较大数的链表
5. if k 为奇数 then $k \leftarrow \lfloor k/2 \rfloor + 1$
6. else $k \leftarrow \lfloor k/2 \rfloor$
7. if $k > 1$ then goto 2
8. $max \leftarrow$ 最大数
9. $second \leftarrow max$ 的链表中的最大

一轮淘汰

继续分
组淘汰



2.4 典型实例

- 时间复杂度分析

命题1 设参与比较的有 t 个元素，经过 i 轮淘汰后元素数至多为 $\lceil t/2^i \rceil$.

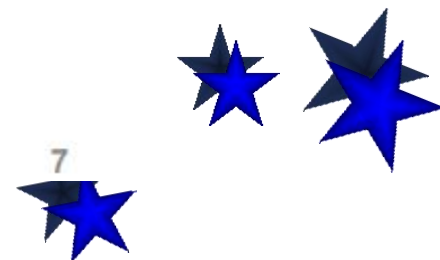
证 对 i 归纳. $i=1$, 分 $\lfloor t/2 \rfloor$ 组, 淘汰 $\lfloor t/2 \rfloor$ 个元素, 进入下一轮元素数是

$$t - \lfloor t/2 \rfloor = \lceil t/2 \rceil$$

假设 i 轮分组淘汰后元素数至多为 $\lceil t/2^i \rceil$,

那么 $i+1$ 轮分组淘汰后元素数为

$$\lceil \lceil t/2^i \rceil / 2 \rceil = \lceil t/2^{i+1} \rceil$$



2.4 典型实例

命题2 *max* 在第一阶段分组比较中总计进行了 $\lceil \log n \rceil$ 次比较.

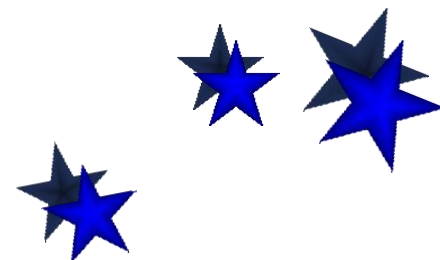
证 假设到产生 *max* 时总计进行 k 轮淘汰, 根据命题 1 有 $\lceil n/2^k \rceil = 1$.

若 $n=2^d$, 那么有

$$k = d = \log n = \lceil \log n \rceil$$

若 $2^d < n < 2^{d+1}$, 那么

$$k = d + 1 = \lceil \log n \rceil$$



2.4 典型实例

第一阶段元素数: n

比较次数: $n-1$

淘汰了 $n-1$ 个元素

锦标赛阶段，找最大值

第二阶段: 元素数 $\lceil \log n \rceil$

比较次数: $\lceil \log n \rceil - 1$

淘汰元素数为 $\lceil \log n \rceil - 1$

从max的淘汰链表中找出最大值

时间复杂度是

$$\begin{aligned} W(n) &= n - 1 + \lceil \log n \rceil - 1 \\ &= n + \lceil \log n \rceil - 2. \end{aligned}$$

空间换时间

